

Bilgisayar Ağları Uygulama Dersi

Hafta 2 Project 1B

Traceroute Hata Yönetimi

Uygulama Föyü

Amaç	Project 1A'da yazdığınız traceroute kodunu gerçek ağ ortamındaki bozuk, eksik, gecikmiş, kopyalanmış veya alakasız paketlere karşı dayanıklı hâle getirmek.
Çalışma dosyası	Önceki hafta kullandığınız traceroute.py dosyası üzerinde geliştirme yapmanız beklenir.

1. Öğrenme hedefleri

- Traceroute uygulamasının neden gerçek ağlarda beklenmedik paketlerle karşılaşabileceğini açıklayabilmek.
- Geçersiz ICMP/UDP/IP paketlerini çökmekten ayıklayabilmek.
- Paket kaybı, sessiz yönlendirici, kopya paket ve yanlış traceroute'tan gelen cevapları doğru yönetebilmek.
- Gereksiz timeout üretmeyen, okunabilir ve modüler bir traceroute tasarlayabilmek.

2. Project 1B'nin özü

Project 1A'da ağın kusursuz çalıştığı varsayılmıştı. Project 1B'de ise bu varsayım bırakılır. Artık ağ; kopya paket, alakasız paket, kayıp paket, sessiz yönlendirici, bozuk başlık, eksik tampon ve yanlış hedefe ait cevap üretebilir. Göreviniz, traceroute kodunuzu bu sorunları görmezden gelmeden ama çökmeye de izin vermeden dayanıklı biçimde geliştirmektir.

Temel ilke

- Her testte asıl soru şudur: Gelen paket gerçekten bu anda gönderdiğiniz probe'a ait ve işlenebilir bir paket mi?
- Cevap hayırsa paketi yok say, ama program çökmemelidir.
- Cevap evetse doğru TTL satırında doğru router/host bilgisini üret.

3. Uygulama mantığı: genel davranış kuralları

TTL başına üç probe: Her TTL değeri için üç probe gönderilir; bu aşamada üç geçerli ve benzersiz cevabı topladığınızda bir sonraki TTL'ye geçebilirsiniz.

Destination görünmezse durma: Hedef host hiç cevap vermeyebilir. Böyle durumda maksimum TTL değerine kadar ilerleyip, hedef hiç görünmeden fonksiyondan dönmeniz gerekir.

Bozuk veya alakasız paketleri yok say: Yanlış protokol, yanlış ICMP type/code, parse edilemeyen içerik, kırılmış buffer veya alakasız UDP cevabı işlenmemelidir.

Kopyaları iki kez sayma: Aynı probe'un ya da aynı cevabın kopyası tekrar gelirse sonuç listesine ikinci kez eklenmemelidir.

Yanlış traceroute cevabını kabul etme: Geçerli görünen bir ICMP paketi bile, farklı bir hedefe gönderilmiş eski bir probe'a ait olabilir; bu durumda da paket yok sayılmalıdır.

4. Gereksiz timeout'ları önleme (Test B1)

Paketleri almak için kullandığımız kodu hatırlayalım:

```
if recvsock.recv_select(): # Check if there's a packet to process.
```

```
    buf, address = recvsock.recvfrom() # Receive the packet.
```

Kodunuzda değişiklik yaparken, gereksiz zaman aşımı hatalarına dikkat etmeniz gerekecek `recv_select` çünkü bunlar kodunuzu yavaşlatacaktır.

Örneğin, bu kodu iki paket almak için kullanırsanız, üçüncü çağrı `recv_select` her zaman zaman zaman aşımına uğrayacaktır (çünkü alınacak üçüncü bir paket yoktur):

```
def read_two_packets_slow():
```

```
    while recvsock.recv_select():
```

```
        recvsock.recvfrom()
```

Bunun aksine, bu kodda her iki çağrı da `recv_select` `True` değerini döndürüyor (aldığınız iki pakete karşılık geliyor) ve zaman aşımı söz konusu değil:

```
def readtwo_packets_fast():
```

```
    if recvsock.recv_select():
```

```
        recvsock.recvfrom()
```

```
    if recvsock.recv_select():
```

```
        recvsock.recvfrom()
```

Alım kuyruğunu tamamen boşaltmaya çalışmak çoğu zaman gereksiz timeout üretir. Basit ve hatasız bir topolojide `recv_select` çağrılarının timeout'a düşmemesi beklenir; çünkü her probe için bir cevap gelecektir.

Bu test, kodunuzun bu basit örnekte bir çağrı sırasında zaman aşımına uğramadığını kontrol eder.

Bunun yerine, paket alımı nedeniyle tüm çağrılar `True` değerini döndürmesi gerekir.

İpuçları

Proje 1A uygulamanız bu testi zaten geçiyor olabilir. Ancak, Proje 1B'ye daha fazla özellik ekledikçe, değişiklikleriniz bu testin başarısız olmasına neden olmaya başlayabilir. Proje üzerinde çalışırken bu testi aklınızda tutmanızı öneririz.

Genellikle, bu testi geçemiyorsanız, bunun nedeni bir sonraki TTL'ye geçmeden önce tüm alma kuyruğunu boşaltmaya çalışmanızdır (yukarıdaki `while` döngüsü gibi). Bu teknik olarak işe yarar, ancak çok yavaş olacaktır.

Bunun yerine, gönderdiğiniz üç sorguya üç yanıt aldıktan sonra bir sonraki TTL'ye geçmeniz gerekiyor.

Ne yapmalısın?

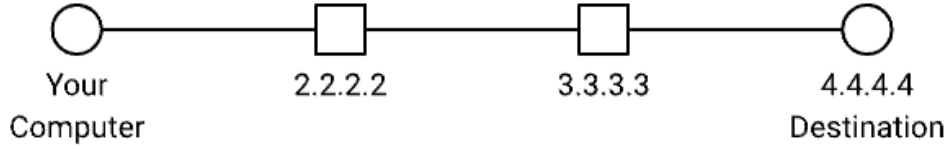
- Her TTL için üç probe gönderdiysen ve bu üç probe'a karşılık üç geçerli cevap aldıysan, fazladan `recv_select` çağırıp kuyruğu sonsuza kadar boşaltmaya çalışma.
- Özellikle `while recv_select()`: şeklindeki kör döngüler, üçüncü paketten sonra dördüncü paketi bekleyip gereksiz gecikme yaratabilir.

5. Geçersiz paketleri yönetme (B2-B8)

Bu testlerde beklenen davranış şudur: geçersiz paketi sessizce yok say, programı çökertme, geçerli paketleri işlemeye devam et.

Bu testler, kodunuzun çökmeden geçersiz paketleri göz ardı ettiğini kontrol eder.

Bu bölümdeki testlerin her biri aynı yapıya sahiptir:



İlk çalıştırma: 3.3.3.3, aldığı ilk iki sorguya geçersiz paket üretiyor.

Giriş hedefi: "4.4.4.4"

Beklenen çıktı: ["2.2.2.2"], ["3.3.3.3"], ["4.4.4.4"]]

İkinci çalıştırma: 3.3.3.3, aldığı üç sorgunun tümüne geçersiz paket gönderiyor.

Giriş hedefi: "4.4.4.4"

Beklenen çıktı: ["2.2.2.2"], [], ["4.4.4.4"]]

Test	Kontrol edilen durum	Kodun yapması gereken	Zorluk
B2	Geçersiz ICMP type	Type değeri "time exceeded" veya "destination unreachable" değilse paketi yok say.	Kolay
B3	Geçersiz ICMP code	Type = time exceeded olsa bile code, "TTL exceeded in transit" değilse paketi yok say.	Kolay
B4	Yanlış IP protocol	IP üstbilgisindeki protocol alanı ICMP değilse, ICMP cevabı gibi davranma.	Kolay
B5	Parse edilemeyen payload	IPv4 paketinin içeriği anlamsız/bozuksa paketi görmezden gel.	Kolay
B6	Eksik/kırılmış buffer	Paket beklenenden kısaysa hata yakala; index/parse hatasıyla çökme.	Kolay
B7	Alakasız UDP cevabı	Router'dan ICMP yerine UDP gelirse bu, traceroute cevabı değildir; yok say.	Orta
B8	IP options var	IPv4 başlık uzunluğu 20 bayttan büyükse seçenek alanını atlayarak paketi yine doğru parse et.	Orta

6. Paket kaybı ve eksik cevaplar (B9-B12)

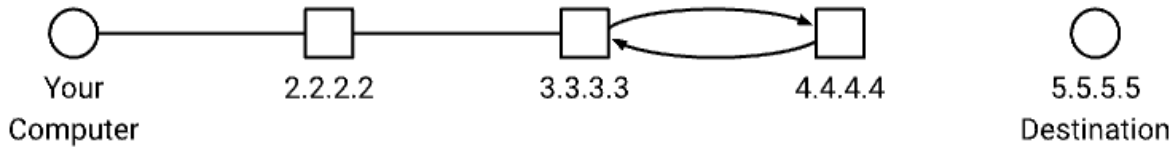
Gerçek internette her probe mutlaka cevap dönmez. Bazen yönlendirici sessizdir, bazen hedef yoktur, bazen paket yolda kaybolur, bazen de rota bir döngüye girer. Traceroute'un sağlam sürümü bu durumları bir hata değil, ağ davranışı olarak ele almalıdır.

Bu testler, kodunuzun ağda paketlerin düşmesi, sunucuların/yönlendiricilerin yanıt göndermeyi reddetmesi veya ağda paketlerin kaybolması durumlarını ele alıp almadığını kontrol eder.

Test B9: Yönlendirici Döngüleri

Yönlendiricilerde arıza varsa, hedefe giden yol bir döngü içerebilir. TTL değerini ne kadar yüksek ayarlarsanız ayarlayın, hedefe giden paketler TTL süresi dolana kadar bu döngüde sıkışıp kalacaktır.

Bu test, yönlendirme döngüsü içeren bir topolojiyi simüle eder. Kodunuz, maksimum TTL'ye ulaşana ve geri dönene kadar (hedefi hiç görmeden) döngüdeki yönlendiricileri tekrar tekrar döndürmelidir.



Topoloji: Bilgisayarınız 1.1.1.1'den 2.2.2.2'ye, oradan 3.3.3.3'e, oradan 4.4.4.4'e ve son olarak 3.3.3.3'e doğru bir döngü oluşturuyor. Hedef (5.5.5.5) hiçbir zaman ulaşılamıyor.

Giriş hedefi: "5.5.5.5"

Beklenen çıktı: ["2.2.2.2"], ["3.3.3.3"], ["4.4.4.4"], ["3.3.3.3"], ["4.4.4.4"], ...]

ve bu işlem, maksimum TTL değeri olan 30'a ulaşana kadar tekrarlanmalıdır.

Test B10: Eksik Ana Bilgisayar

Bazen hedef IP adresi traceroute sorgunuza yanıt vermeyebilir. Bunun nedeni şunlar olabilir:

- Hedef IP adresi mevcut değil.
- Hedef makine, traceroute sorgularına yanıt vermeyecek şekilde yapılandırılmıştır.
- Hedefe giden yol boyunca, traceroute sorgu paketini engelleyen bir ağ güvenlik duvarı bulunmaktadır.

Bu test, hedef noktanın mevcut olmadığı bir topolojiyi simüle eder. Kodunuz, maksimum TTL'ye ulaşana kadar (hedef noktayı hiç görmeden) yoklama göndermeye devam etmeli ve ardından geri dönmelidir.



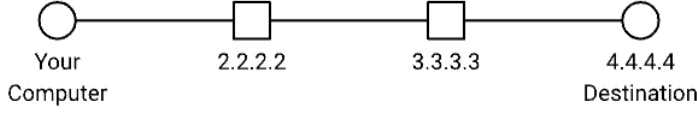
Giriş hedefi: "5.5.5.5" (mevcut değil)

Beklenen çıktı: ["2.2.2.2"], ["3.3.3.3"], ["4.4.4.4"], [], [], [], ...]

4.4.4.4'e ulaşan herhangi bir traceroute sorgusu düşürülür, bu nedenle dönüş değeri 4'ten 30'a kadar tüm TTL'ler için boş listeler içermelidir.

Test B11: Sessiz Yönlendiriciler

İnternetteki bazı yönlendiriciler, çeşitli nedenlerden dolayı traceroute sorgularına hiç yanıt vermez. Bu test, traceroute sorgularına yanıt vermeyi reddeden bir yönlendiriciyi simüle eder.



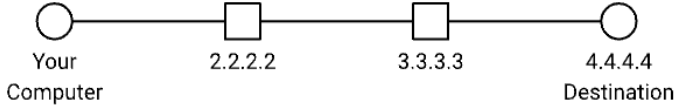
Giriş hedefi:"4.4.4.4"

Beklenen çıktı:[["2.2.2.2"], [], ["4.4.4.4"]]

3.3.3.3 yönlendiricisi tüm süre boyunca sessiz kalıyor ve traceroute sorgularınıza yanıt vermiyor.

B12 Testi: Ara sıra Düşüşler

Bazen internette paketler kaybolabiliyor. Bu test, paketlerin kaybolmasını simüle eder. Kodunuzun bakış açısından, bu durum temelde yönlendiricinin ICMP sorgularına yanıt vermeyi reddetmesiyle aynıdır.



İlk çalıştırma: TTL 2 ile (3.3.3.3'ü hedefleyen) yapılan ilk iki deneme, 3.3.3.3'e ulaşmadan önce başarısız oldu.

- Giriş hedefi:"4.4.4.4"
- Beklenen çıktı:[["2.2.2.2"], ["3.3.3.3"], ["4.4.4.4"]]

İkinci çalıştırma: TTL 2 (3.3.3.3'ü hedefleyen) ile yapılan tüm denemeler, 3.3.3.3'e ulaşmadan önce iptal edildi.

- Giriş hedefi:"4.4.4.4"
- Beklenen çıktı:[["2.2.2.2"], [], ["4.4.4.4"]]

Test	Kontrol edilen durum	Kodun yapması gereken	Zorluk
B9	Router loop	TTL artsa bile hedefe gidilemez. Döngüde görünen router'ları maksimum TTL'e kadar tekrar tekrar döndür.	Kolay
B10	Missing host	Hedef IP hiç yoksa veya hiç cevap vermiyorsa maksimum TTL'e kadar devam et; hedef görünmeden dön.	Orta
B11	Silent routers	Aradaki router cevap vermezse o TTL için boş liste üret; sonraki TTL'lerde cevap gelirse devam et.	Orta

Test	Kontrol edilen durum	Kodun yapması gereken	Zorluk
B12	Occasional drops	Kayıp paketleri sessiz router gibi düşün. Bazı probe'lar kaybolursa da eldeki geçerli cevaplarla ilerle.	Kolay

Önemli yorum

- Bir TTL'de hiç geçerli cevap alamaman, traceroute'un tamamen başarısız olduğu anlamına gelmez.
- Sonuç listesinde o TTL için [] üretip sonraki TTL'ye geçebilmelisin.
- Hedef hiç görünmese bile fonksiyon sonlanmalı; sonsuz döngüye girmemelisin.

7. Yinelenen paketler (B13-B15)

Bu bölüm proje içindeki en zor kısımlardan biridir. Sorun yalnızca 'aynı IP'yi bir daha ekleme' değildir. Asıl sorun, kopya probe veya kopya cevabın alım kuyruğunda zaman kaymasıyla görünmesidir.

Test B13: Yinelenen Yanıtlar

Bazen tek bir paket gönderirsiniz, ancak yanıt iki kez gelir. Bu iki paketten gelen yanıtı dikkate almamalısınız.

Bu testte, TTL 2 ile (3.3.3.3'e yönelik) bir yoklama gönderdiğinizde, 3.3.3.3'ten gelen yanıt iletim sırasında ikiye katlanıyor.

Test B14: Çift Probeler

Bazen bir paket gönderirsiniz ve gönderdiğiniz paket kopyalanır. Bu, yönlendiricinin paketinizin iki kopyasını almasına ve iki kez yanıt vermesine neden olur. İkinci yanıtı dikkate almamalısınız.

Bu testte, TTL 2 ile (3.3.3.3'ü hedefleyerek) bir probe gönderdiğinizde, probe kopyalanır. Bu durum, 3.3.3.3'ün birden fazla ICMP yanıtı üretmesine neden olur.

Test B15: Gecikmeli Kopyalar

Bazen paketler (sorgulama veya yanıtlar) kopyalanabilir ve ciddi gecikmelere yol açabilir.

Sonuç olarak, 3.3.3.3 adresini hedefleyen bir sorgu gönderdiğinizde, 2.2.2.2 adresini hedefleyen önceki bir sorgudan gelen aynı yanıtı alabilirsiniz.

Veya, 2.2.2.2 adresine yönelik bir sorgu gönderildiğinde, bu sorgu kopyalanabilir. Daha sonra, 3.3.3.3 adresine yönelik bir sorgu gönderdiğinizde, 2.2.2.2 adresine yönelik kopyalanmış sorgudan bir yanıt alabilirsiniz.

Bu testte, sorgular ve yanıtlar tekrarlanabilir ve iletim sırasında ciddi gecikmeler yaşanabilir. Bu durum, belirli bir yönlendiriciden gelen yanıt sorgusunun, önceki yönlendiricilerden gelen yinelenen yanıtlarla veya önceki yönlendiricilere gönderilen yinelenen sorgulara verilen yanıtlarla öncelenmesine neden olabilir.

Test	Kontrol edilen durum	Kodun yapması gereken	Zorluk
B13	Duplicate responses	Tek bir probe'a ait cevabın ağda kopyası oluşursa yalnızca bir tanesini kabul et.	Zor
B14	Duplicate probes	Gönderdiğiniz probe'un kopyası oluşup router iki kez	Zor

Test	Kontrol edilen durum	Kodun yapması gereken	Zorluk
		cevap verirse ikinci cevabı yok say.	
B15	Delayed duplicates	Eski TTL'lere ait gecikmiş kopya cevaplar sonraki TTL sırasında gelebilir; yalnızca o anki probe ile eşleşen cevapları kabul et.	Zor

Neden zor?

- Sadece kaynak IP'ye bakmak yetmez; çünkü aynı router'dan farklı probe'lara ait cevaplar gelebilir.
- Bir TTL'de üç benzersiz cevap aldın diye kuyruğu tamamen görmezden gelersen, kuyruğun içinde kalan kopyalar sonraki TTL'yi bozabilir.
- Bu yüzden 'hangi cevabın hangi probe'a ait olduğunu' ayırt edecek bir eşleme mantığı tasarlamam gerekir.

8. Yanlış traceroute'tan gelen cevap (B16)

Test B16: Alakasız TTL Yanıtı

Geçerli bir ICMP zaman aşımı paketi alırsanız, ancak paket kodunuzun göndermediği farklı bir hedefe yönelik bir sorguya yanıt olarak oluşturulmuşsa, paketi yok saymalısınız.

Bu durum, örneğin, farklı bir hedef için yapılan önceki bir traceroute çalıştırmasından gelen bir paketin ciddi şekilde gecikmesi ve bunun yerine bu traceroute çalıştırması tarafından alınması durumunda meydana gelebilir.

Topoloji: Bilgisayarınızdan 2.2.2.2'ye, oradan 3.3.3.3'e, oradan da 4.4.4.4'e (Hedef)

İlk çalıştırma: 3.3.3.3, aldığı ilk iki sorguya geçersiz paket üretiyor.

Giriş hedefi:"4.4.4.4"

Beklenen çıktı:[["2.2.2.2"], ["3.3.3.3"], ["4.4.4.4"]]

İkinci çalıştırma: 3.3.3.3, aldığı üç sorgunun tümüne geçersiz paket gönderiyor.

Giriş hedefi:"4.4.4.4"

Beklenen çıktı:[["2.2.2.2"], [], ["4.4.4.4"]]

- Bu yüzden yalnızca paket biçiminin geçerli olması yetmez; paketin, o anda gönderdiği probe ile ilişkilendirilebilmesi gerekir.
- Özellikle gömülü orijinal IP/UDP başlıkları incelenerek hedef IP ve ilgili probe bilgileri doğrulanmalıdır.
- Doğrulanamayan paket işlenmemelidir.

9. Önerilen tasarım yaklaşımı

Çok sayıda iç içe geçmiş if bloklarıyla paket işleme akışını yönetmeyin. Bunun yerine, küçük doğrulama fonksiyonları yazın ve bir paketi ancak bütün kontrollerden geçerse işleyin.

Yardımcı fonksiyon fikri	Ne işe yarar?
parse_ipv4_packet(buf)	Paket boyunu, başlık uzunluğunu ve temel alanları güvenli şekilde çözer; eksik/bozuk paketleri ayıklar.
is_valid_icmp_response(pkt)	ICMP type/code ve IP protocol alanlarını denetler.
extract_embedded_probe_info(pkt)	ICMP hata paketinin içinde taşınan orijinal IP/UDP bilgilerini çıkarır.
matches_current_probe(info, ttl, dest, port)	Gelen cevabın bu anda gönderdiği probe'a ait olup olmadığını belirler.
record_unique_response(...)	Aynı cevabın veya kopya probe cevabının ikinci kez sonuç listesine girmesini önler.

10. Temiz kod ve hata ayıklama notları

- Deneme-yanılma ile ilerleme. Her satırın ne yaptığını gerçekten anla.
- Kod karışmaya başladığında refactor et. Okunabilirlik arttıkça mantık hataları daha kolay görünür.
- Tek bir testi düzeltirken daha önce geçen bir testi bozmadığından emin ol.
- Network programlama yerelde her zaman tekrar üretilemez; bu yüzden bazen en güçlü araç dikkatli kod okuma ve zihinsel simülasyondur.
- Özellikle duplicate testlerinde, alım kuyruğunda kalan eski cevapların sonraki TTL'yi bozmaması için tasarımı baştan düşünmek gerekir.